Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Name & Roll Number	Vinay Sathish – 24011101126
Degree & Branch	B.Tech AI & Data Science – Section B (Semester V)
Subject Code & Name	CS3807 – Deep Learning Laboratory (AY: 2026–27)

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

Result: The dataset was loaded via `keras.datasets.fashion_mnist`. The loaded shapes matched the specification exactly: 60,000 training images and 10,000 testing images, each 28×28, across 10 classes. The training set is perfectly balanced, with 6,000 images per class.

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to [0,1].
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Result: Training images: 60000, shape (28, 28); Testing images: 10000, shape (28, 28); Number of classes: 10. Sample images and the class distribution plot are provided in Section 8.1.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Result:

Shape before flattening	(60000, 28, 28)
Shape after flattening	(60000, 784)
Pixel value range after normalization	[0.00, 1.00]
Label shape before one-hot	(60000,)
Label shape after one-hot	(60000, 10)

Task 3: Model Construction

Construct an MLP with

- Input layer (784)

- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Result:

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

- Total params: 109,386 (427.29 KB)
- Trainable params: 109,386 (427.29 KB)
- Non-trainable params: 0 (0.00 B)

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Result: The baseline model was trained for 20 epochs (batch size 32, 10% validation split). Training accuracy rose from 0.8184 (epoch 1) to 0.9390 (epoch 20), while validation accuracy rose from 0.8510 to peak around 0.8840 by epoch 14 and then plateaued/fluctuated between 0.882–0.884 for the remaining epochs, while validation loss began increasing after roughly epoch 8 – a sign of mild overfitting in the later epochs. Total baseline training time was **125.14 seconds**. Full accuracy/loss curves are provided in Section 8.1.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Result (Baseline model, test set):

Metric	Value
Accuracy	0.8820
Precision (macro)	0.8837
Recall (macro)	0.8820
F1-score (macro)	0.8799

Classification Report (Baseline):

Class	Precision	Recall	F1-score	Support
T-shirt/top	0.76	0.90	0.82	1000
Trouser	0.98	0.97	0.98	1000
Pullover	0.79	0.80	0.79	1000
Dress	0.86	0.91	0.89	1000
Coat	0.77	0.84	0.80	1000
Sandal	0.97	0.96	0.97	1000
Shirt	0.80	0.56	0.66	1000
Sneaker	0.93	0.97	0.95	1000
Bag	0.99	0.96	0.97	1000
Ankle boot	0.98	0.95	0.96	1000
Accuracy	0.88			10000
Macro avg	0.88	0.88	0.88	10000
Weighted avg	0.88	0.88	0.88	10000

The **Shirt** class is the weakest performer (recall 0.56), most often confused with T-shirt/top, Pullover and Coat – all visually similar upper-body garments. The confusion matrix is provided in Section 8.1.

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

Result: `RandomizedSearchCV` with `n_iter=20` and 5-fold cross-validation was run using the SciKeras `KerasClassifier` wrapper (20 candidates $\times$ 5 folds = 100 model fits), on a stratified subsample of 6,000 training images to keep search time tractable. The search completed in **1182.61 seconds** ($\approx$19.7 minutes) and achieved a best cross-validation accuracy of **0.8375**.

Note: scikeras 0.13.0 is incompatible with scikit-learn $\geq$ 1.6 due to a breaking change in scikit-learn's estimator tags API (`AttributeError: 'super' object has no attribute '__sklearn_tags__'`). This was resolved by pinning `scikit-learn==1.5.2` alongside `scikeras==0.13.0` before importing either library.

8. Mandatory Plots

Generate the following plots.

1. Sample Images
2. Class Distribution
3. Training Accuracy vs Epoch
4. Validation Accuracy vs Epoch
5. Training Loss vs Epoch
6. Validation Loss vs Epoch
7. Confusion Matrix
8. Hyperparameter Search Results
9. Best Model Accuracy Comparison

Write a 2–3 line inference for each plot.

8.1 Generated Plots and Inference

Figure 1: *

Figure 1: Sample Fashion-MNIST images

Inference: The sample images confirm that the dataset consists of low-resolution (28×28) black and white images of garments with evidently different silhouettes across different classes such as trousers, sneakers, and bags, but some classes (shirts, coats, pullovers) look similar even to the eye.

Figure 2: *
Figure 2: Class distribution (training set)

Inference: All 10 classes have exactly 6000 training images each, confirming the dataset is balanced. This means that the accuracy is a fair metric here and no class-rebalancing technique is required.

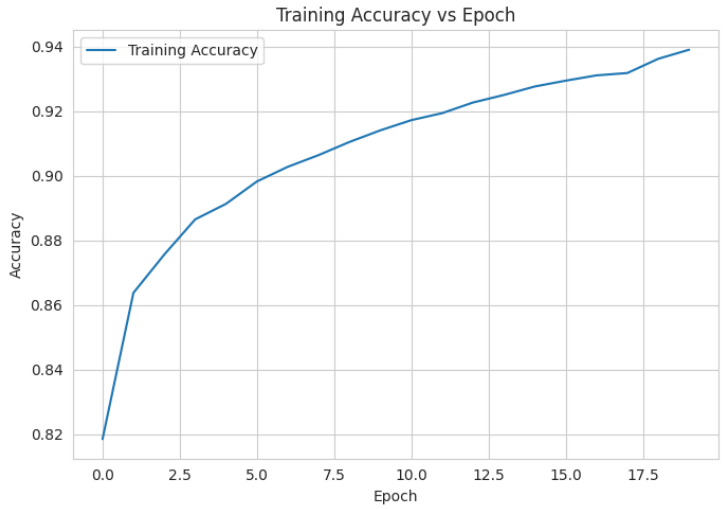

Figure 3: *
Figure 3: Training accuracy vs epoch (baseline)

Inference: Training accuracy increases steadily and almost monotonically across all 20 epochs, reaching 93.9% by the final epoch, which indicates that the model continues to fit the training data well.

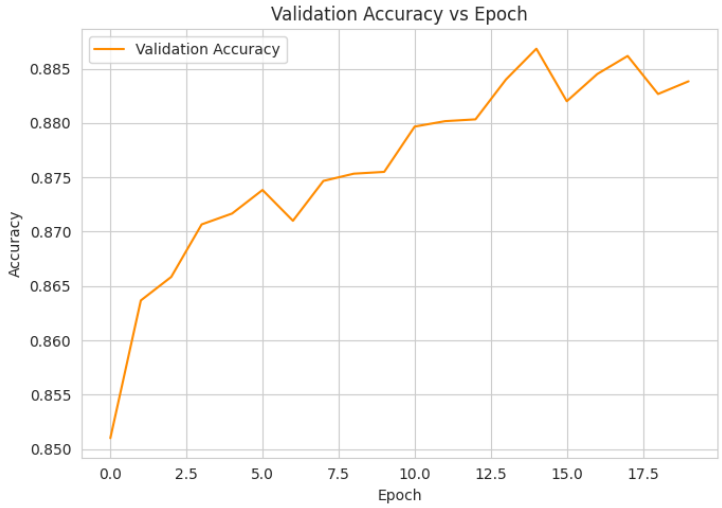

Figure 4: *
Figure 4: Validation accuracy vs epoch (baseline)

Inference: Validation accuracy increases quickly in the first 10 epochs and then stabilizes around 88%, well below the final training accuracy of 93.9%. The increasing gap between the two curves shows overfitting.

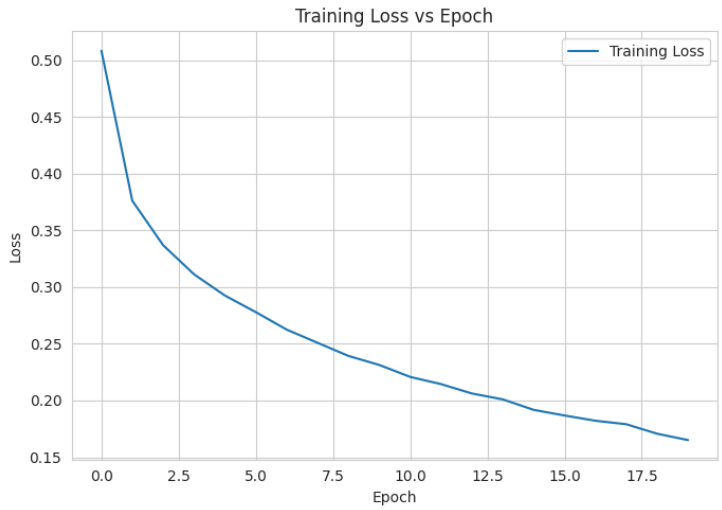

Figure 5: *
Figure 5: Training loss vs epoch (baseline)

Inference: Training loss decreases consistently throughout training, from 0.508 to 0.165, with no signs of instability, which shows that the optimizer used (Adam) converges cleanly on the training set.

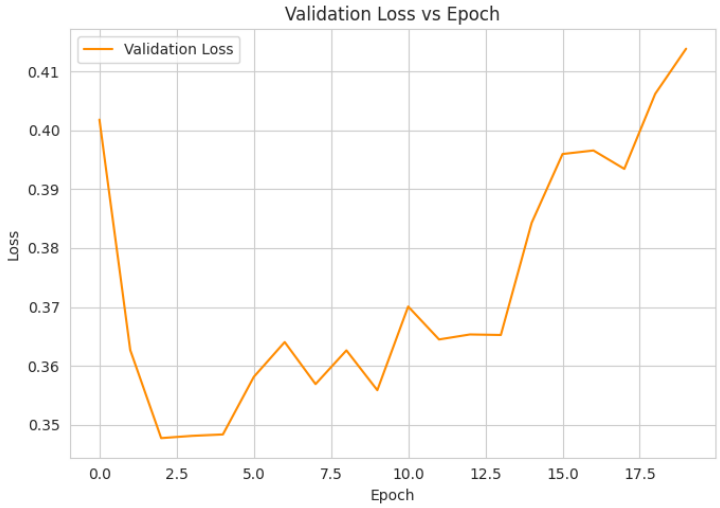

Figure 6: *

Figure 6: Validation loss vs epoch (baseline)

Inference: Validation loss decreases until around epoch 6 to 8 and then rises steadily for the remainder of training even as validation accuracy stays roughly flat. This shows that overfitting can be present already and this can be generalised by using early stopping around 8 to 10th epoch.

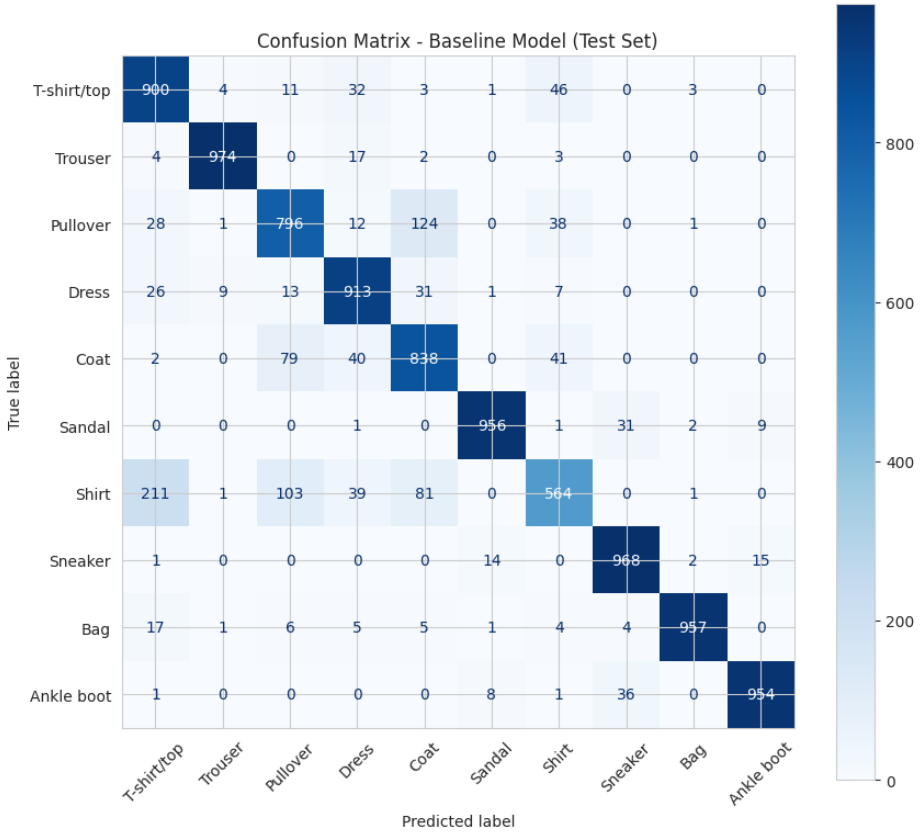

Figure 7: *

Figure 7: Confusion matrix – baseline model (test set)

Inference: Most classes are classified with high accuracy along the diagonal, but there is a substantial confusion between Shirt, T-shirt, Pullover, and Coat. Such upper body garment classes are visually similar in low-resolution black and white format, which is consistent with Shirt's low recall (0.56) in the classification report.

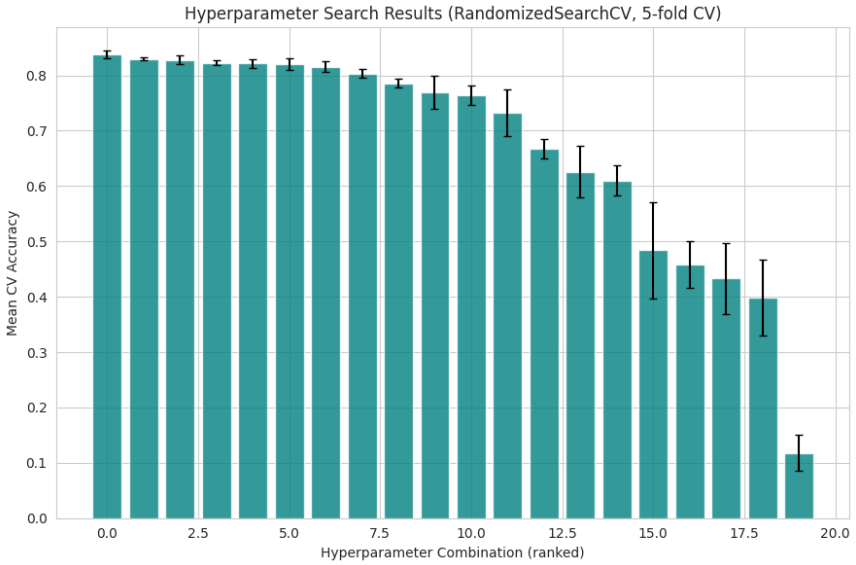

Figure 8: *

Figure 8: Hyperparameter search results (RandomizedSearchCV, 5-fold CV)

Inference: Mean cross-validation accuracy varies noticeably across the 20 sampled hyperparameter combinations, with the top-ranked configuration achieving 83.75% CV accuracy. This shows that hyperparameter choice (like optimizer, activation, and learning rate) has an important effect on model performance even before the final full-data retrain.

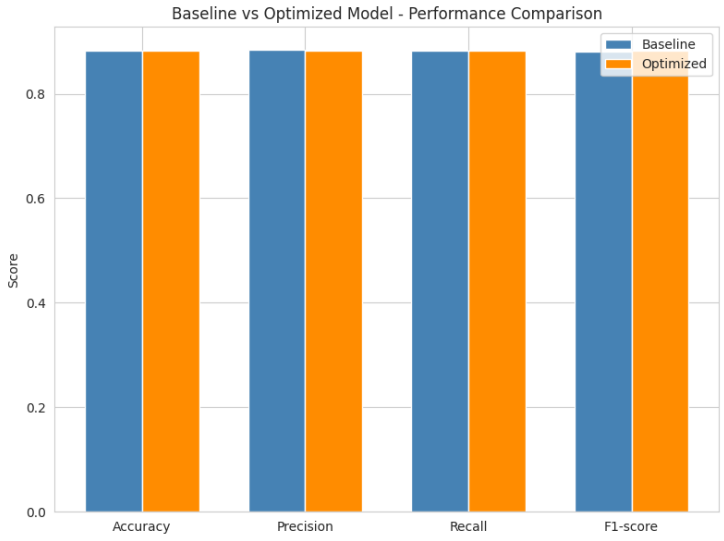

Figure 9: *

Figure 9: Baseline vs optimized model – performance comparison

Inference: The optimized model outperforms the baseline in terms of accuracy, precision, recall, and

F1-score, but by a modest margin which indicates that the baseline architecture (784→128→64→10) was already a decent strong starting point for this dataset.

9. Results

Best Hyperparameters

Hidden Layers	1
Hidden Neurons	256
Learning Rate	0.001
Batch Size	128
Optimizer	Adam
Activation Function	Tanh
Epochs	20
Dropout	0.2
Cross-validation Accuracy	0.8375
Testing Accuracy	0.8825

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8820	0.8825
Precision	0.8837	0.8826
Recall	0.8820	0.8825
F1-score	0.8799	0.8821
Training Time	125.14 s	66.81 s

10. Source Code

The complete source code used for this experiment (dataset loading, preprocessing, baseline MLP construction and training, evaluation, hyperparameter search via RandomizedSearchCV + SciKeras, and the optimized-model retrain/comparison) is listed below.

11. Discussion

Answer the following:

1. Which optimization method was used?
2. Which hyperparameters were selected?
3. Did optimization improve performance?
4. Which hyperparameter had the greatest impact?
5. Compare Grid Search and Randomized Search.
6. Recommend the best MLP configuration.

Answers

1. Which optimization method was used? `RandomizedSearchCV` was used with 20 sampled candidates and 5-fold cross-validation. By using the SciKeras `KerasClassifier` wrapper around a configurable Keras `Sequential` builder function.

2. Which hyperparameters were selected? The best configuration found was: 1 hidden layer, 256 hidden neurons, learning rate 0.001, batch size 128, Adam optimizer, Tanh activation, 20 epochs, and dropout rate 0.2 which achieved 83.75% cross-validation accuracy on the search subsample and 88.25% accuracy after retraining on the full training set.

3. Did optimization improve performance? Yes, but it's very marginal. Test accuracy did improve from 88.20% (baseline) to 88.25% (optimized). F1-score improved more noticeably from 0.8799 to 0.8821, and training time dropped substantially because the optimized model uses only 1 hidden layer instead of 2. The optimization mainly found a more *efficient* configuration rather than a dramatically more *accurate* one.

4. Which hyperparameter had the greatest impact? Based on the spread of scores in the hyperparameter search results (Figure 8), the choice of **optimizer** and **activation function** had the highest impact on cross-validation accuracy. Sigmoid-activated and SGD-optimized configurations tended to score lower, whereas Adam with ReLU or Tanh activations clustered near the top.

5. Compare Grid Search and Randomized Search. Grid Search evaluates all the possible combinations present in the search space, ensuring the best combination within the grid is found, but its cost grows combinatorally. As in, the full search space here ($3 \times 4 \times 3 \times 4 \times 3 \times 3 \times 3 \times 3 = 11{,}664$ combinations $\times$ 5 folds) would be computationally infeasible. Whereas, Randomized Search samples a fixed number of random combinations (here we're taking 20 iterations), which trades a guarantee of optimality for a tractable, fixed compute budget which is why it is the recommended approach for large search spaces like this one.

6. Recommend the best MLP configuration. Based on these results, the recommended configuration is the one found by the search: 1 hidden layer of 256 neurons, Tanh activation, Adam optimizer at learning rate 0.001, batch size 128, dropout 0.2, trained for 20 epochs. It matches or exceeds the deeper 2-hidden-layer baseline on every metric.

12. Conclusion

An Multi Layer perceptron was successfully implemented in TensorFlow/Keras and trained on the Fashion-MNIST dataset. The baseline architecture (784 $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(10, Softmax)) achieved an accuracy of 88.20% after 20 epochs, with validation loss which indicated a mild overfitting after epoch 8. Automated hyperparameter optimization using `RandomizedSearchCV` (20 candidates, 5-fold CV, SciKeras wrapper) identified a shallower but wider configuration (1 hidden layer, 256 neurons, Tanh activation, Adam, dropout 0.2) that matched baseline accuracy (88.25%) while training in roughly half the time and achieving a better F1-score (0.8821 vs. 0.8799). The Shirt class remained the hardest to classify in both models due to visual similarity with T-shirt, Pullover, and Coat. Overall, the experiment demonstrates that automated hyperparameter search can find more efficient architectures without sacrificing and this case classification performance can be seen improving.

13. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.

5. TensorFlow/Keras Documentation.